

BOIDS IN C AND OPENMP

Parallelizing particle simulation

H.Kirollos

<https://github.com/alonzo-bazaar/parallel-programming>

CONTENTS

1. Boids
2. Technologies used
3. Benchmarks

Boids

- Boids (short for bird-oids, or just birds pronounced with a new york accent) consists of many bird-like (hence the name) objects moving together
- Every boid has a position, and a velocity vector, the immediate behaviour of one boid is determined solely by its current state and by the states of nearby boids
- In the scope of one simulation tick all boid behaviours are computed as if in parallel

We have implemented a simpler “force field based” approach compared to the original boids paper, this simpler approach has the same parallel characteristics, it just lacks the more physics based bird simulation of the original paper. The procedure is as follows

- For all boids x
 - If it's too close to the area edge or outside of the area, add a force to going back to the area center
 - For all other boids y ($O(n^2)$)
 - If y is too close to x , add a repelling force to x so they don't collide
 - If y is close enough to x add it as a neighbour of x
 - Add a force to x to move its position closer to the average position of its neighbours
 - Add a force to x to move its velocity closer to the average velocity of its neighbours

The original boids paper goes over several approaches for combining these forces (there called “potentials”, as they act in a more control theory environment), we have opted for a naive weighted sum, followed by clamping of the boid’s velocity.

Every simulation tick in our boids programs is structured as follows

- We have two fixed size buffers of boids (number of boids stays fixed throughout simulation)
 - One is to contain the current state of the simulation
 - One is for writing the state of the simulation in the following tick
- For all boids x in the first buffer (hic est parallelism)
 - Compute next state of x
 - Write next state of x in second buffer
- Swap buffers for next iteration

Implementation

- Program has been developed using C and the OpenMP library/compiler extension
- Sequential and parallel versions of the program use the same code and just turn OpenMP `#pragmas` on or off by changing passed compiler flags
 - (this produces compilation warnings for unrecognized pragmas but these warnings are of little concern)
- Benchmark and graphical versions for the program have been developed, the graphical versions use `raylib` to handle the graphics
- Two versions of simulation code have been developed
 - one where the data is structured in an AoS (array of structs) manner
 - one where the data is structured in an SoA (struct of arrays) manner

Benchmarks and Results

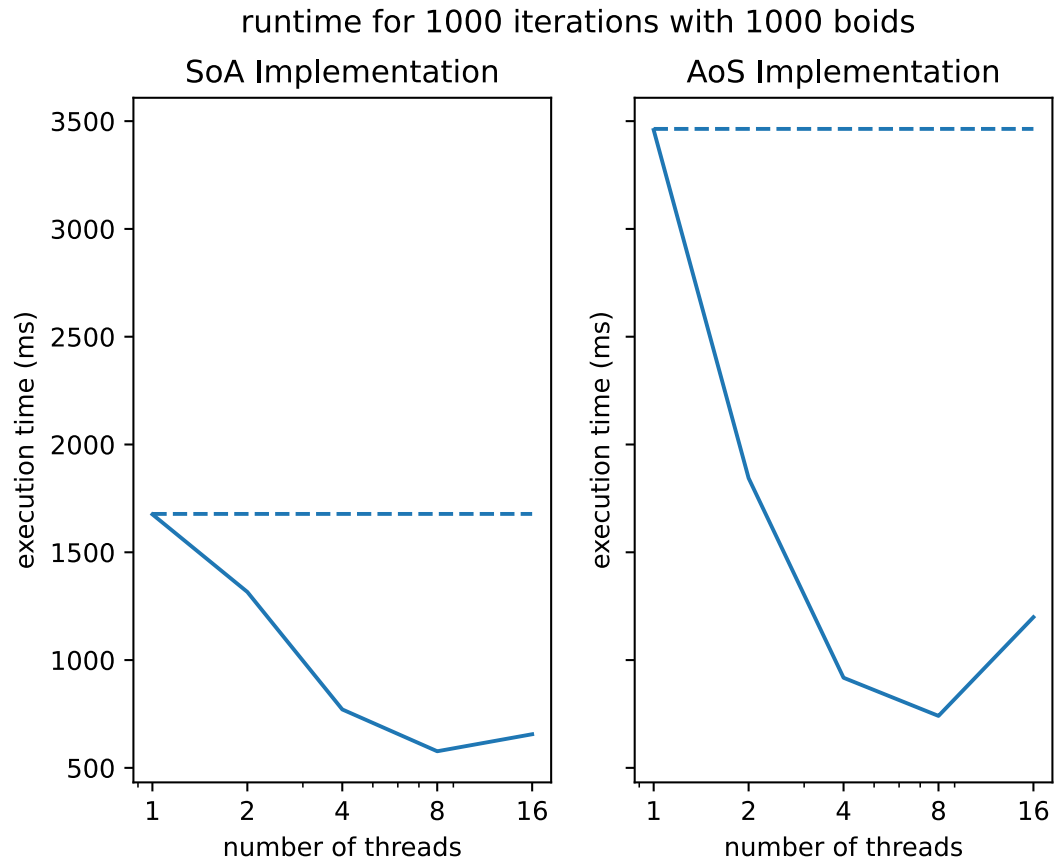
Benchmarks have been run using

- 1000 boids
- 100 warmup iterations
- 1000 timed iterations

Given these parameters, runtime has been measured for different numbers of openmp threads, namely

- 1, 2, 4, 8, and 16

- continuous line parallel execution time as threads increase
- dotted line execution time of sequential version



(the test machine has 8 cores)

- Adding threads reduces runtime as long as thread number does not exceed number of machine cores, after which increasing threads adds overhead
- Parallel execution with only one thread matches time of sequential execution, meaning synchronization overhead in 1 thread case may be considered negligible (given test configuration)
- SoA version of simulation loop scores better than AoS version of the simulation loop for all tried sizes